Lab 8 | 12 March 2026

↳ 15 cells hidden

Lab 9 | 19 March 2025

Handwritten Digit Recognition using CNN

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, D
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
```

```
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-k
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolution
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

Model: "sequential"

Layer (type)	Output Shape	P
conv2d (Conv2D)	(None, 26, 26, 32)	
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	
flatten (Flatten)	(None, 1600)	
dense (Dense)	(None, 128)	2
dense_1 (Dense)	(None, 10)	

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 59s 66ms/step - accuracy: 0.9497 - loss
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 75s 58ms/step - accuracy: 0.9839 - loss
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9888 - loss
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 51s 60ms/step - accuracy: 0.9921 - loss
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 49s 58ms/step - accuracy: 0.9943 - loss
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 59ms/step - accuracy: 0.9952 - loss
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 49s 58ms/step - accuracy: 0.9956 - loss
```

```
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 59ms/step - accuracy: 0.9972 - loss
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9975 - loss
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 59ms/step - accuracy: 0.9971 - loss
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9902 - loss:
Test accuracy: 0.9901999831199646
```

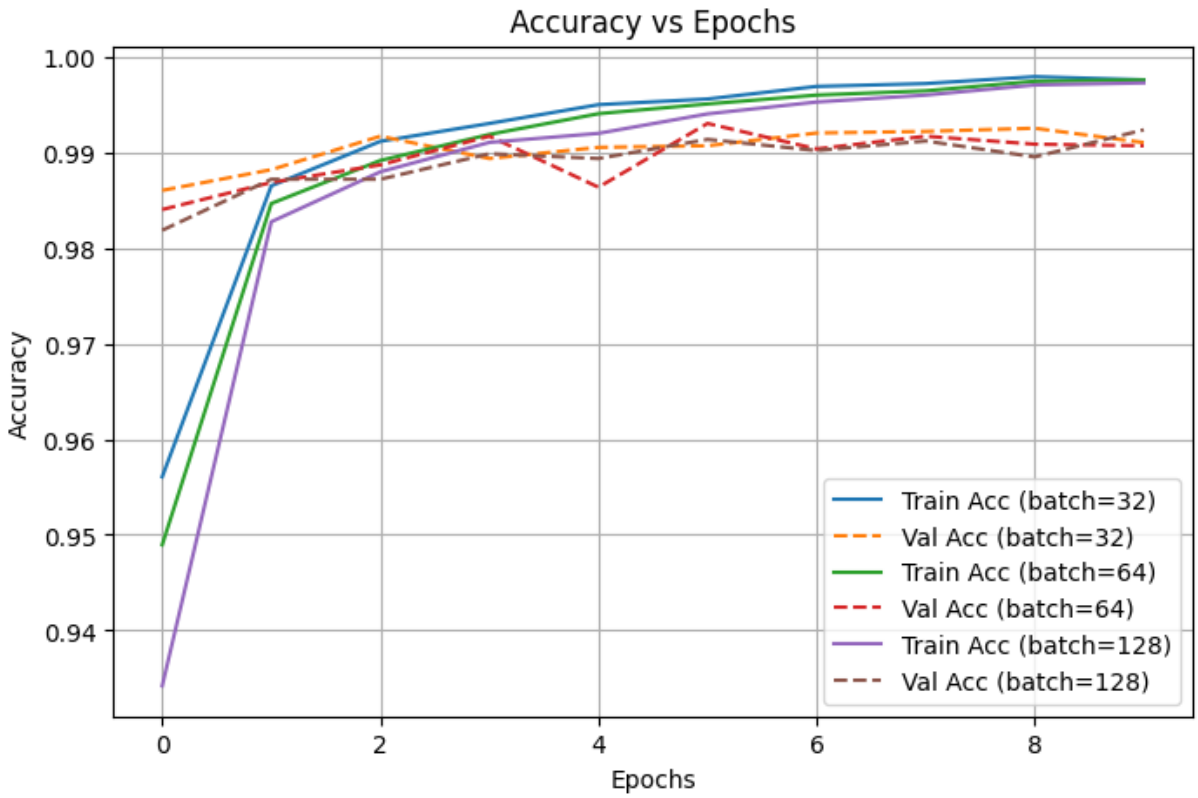
ADDED DIFFERENT BATCHES

```
import matplotlib.pyplot as plt

batch_sizes = [32, 64, 128]
histories = {}

for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1))
        MaxPooling2D((2,2)),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
    history = model.fit(X_train_cnn, y_train,
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
```

```
    plt.plot(histories[batch].history['accuracy'], label=f'Train Ac
    plt.plot(histories[batch].history['val_accuracy'], linestyle='-

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Training with batch size: 32
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 68s 37ms/step - accuracy: 0.9561 - lo
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 74s 32ms/step - accuracy: 0.9865 - lo
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 54s 32ms/step - accuracy: 0.9911 - lo
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 53s 32ms/step - accuracy: 0.9930 - lo
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 53s 31ms/step - accuracy: 0.9950 - lo
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 52s 31ms/step - accuracy: 0.9956 - lo
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 61s 36ms/step - accuracy: 0.9969 - lo
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 73s 31ms/step - accuracy: 0.9972 - lo
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 53s 31ms/step - accuracy: 0.9979 - lo
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 52s 31ms/step - accuracy: 0.9976 - lo

Training with batch size: 64
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 50s 58ms/step - accuracy: 0.9490 - loss
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 47s 55ms/step - accuracy: 0.9846 - loss
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 56ms/step - accuracy: 0.9891 - loss
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 45s 54ms/step - accuracy: 0.9919 - loss
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 55ms/step - accuracy: 0.9940 - loss
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 81s 54ms/step - accuracy: 0.9950 - loss
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 46s 54ms/step - accuracy: 0.9960 - loss
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 50s 59ms/step - accuracy: 0.9964 - loss
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 50s 59ms/step - accuracy: 0.9974 - loss
Epoch 10/10
```

```
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 60ms/step - accuracy: 0.9975 - loss

Training with batch size: 128
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 49s 112ms/step - accuracy: 0.9342 - los
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 46s 108ms/step - accuracy: 0.9827 - los
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 45s 106ms/step - accuracy: 0.9879 - los
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 47s 111ms/step - accuracy: 0.9910 - los
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 110ms/step - accuracy: 0.9920 - los
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 80s 106ms/step - accuracy: 0.9940 - los
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 106ms/step - accuracy: 0.9952 - los
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 103ms/step - accuracy: 0.9960 - los
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 103ms/step - accuracy: 0.9970 - los
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 44s 104ms/step - accuracy: 0.9972 - los
```

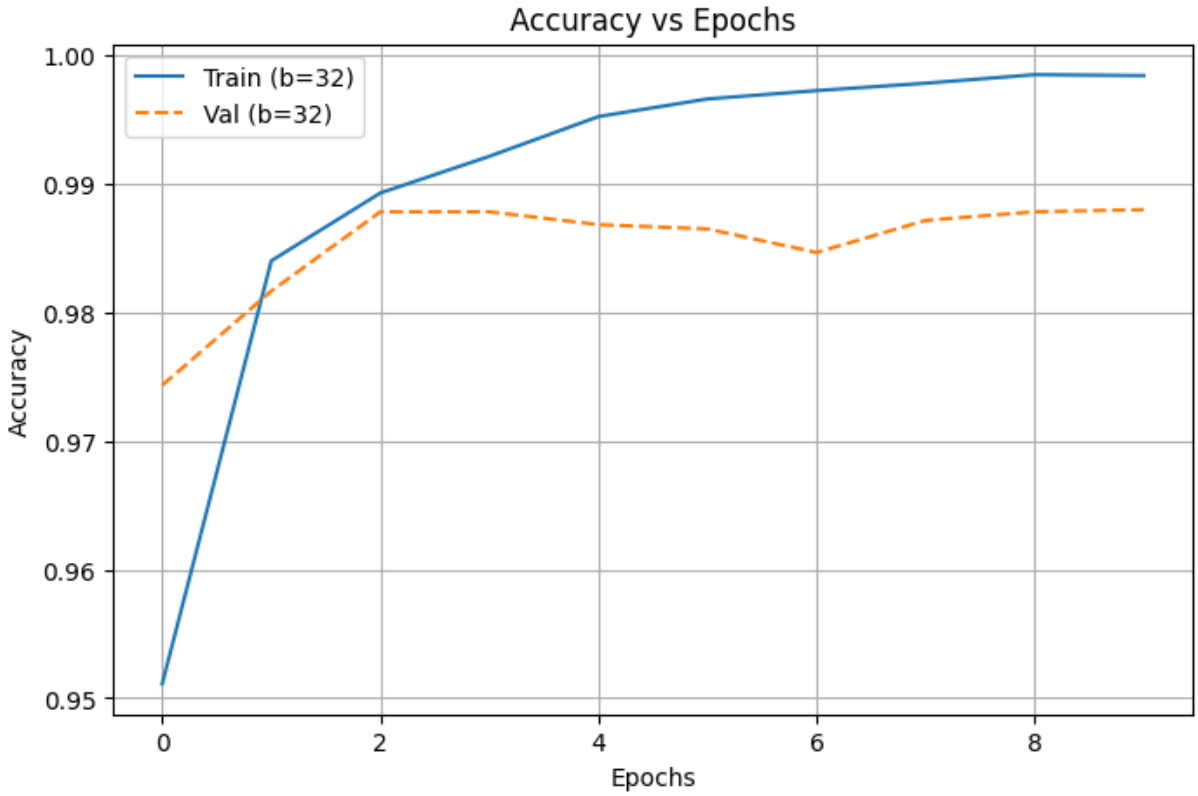


Remove 1 convolutional layer

```
import matplotlib.pyplot as plt

batch_sizes = [32]
histories = {}

for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1))
        MaxPooling2D((2,2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
    history = model.fit(X_train_cnn, y_train,
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Train (b
    plt.plot(histories[batch].history['val_accuracy'], linestyle='-

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Training with batch size: 32
Epoch 1/10
```

```
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 54s 30ms/step - accuracy: 0.9511 - lo
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 71s 23ms/step - accuracy: 0.9840 - lo
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 36s 22ms/step - accuracy: 0.9893 - lo
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 39s 23ms/step - accuracy: 0.9921 - lo
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 39s 23ms/step - accuracy: 0.9952 - lo
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 40s 22ms/step - accuracy: 0.9966 - lo
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 37s 22ms/step - accuracy: 0.9973 - lo
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 42s 23ms/step - accuracy: 0.9978 - lo
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 40s 22ms/step - accuracy: 0.9985 - lo
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 38s 23ms/step - accuracy: 0.9984 - lo
```



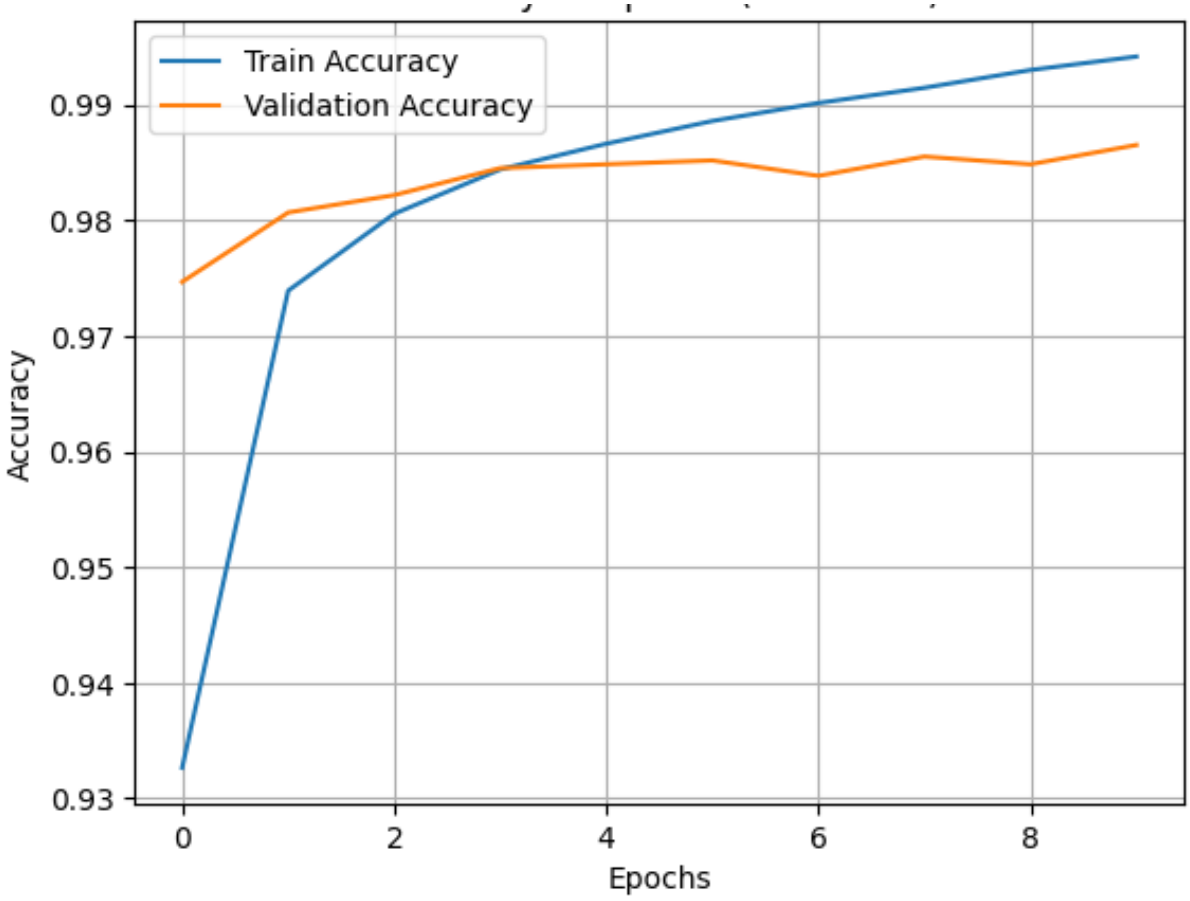
Remove 1 dense layer

```
import matplotlib.pyplot as plt
```

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(10, activation='softmax')   # only one dense layer now
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train_cnn, y_train,
                    epochs=10,
                    batch_size=32,
                    validation_split=0.1)

# Plot accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accurac

plt.title("Accuracy vs Epochs (Batch=32)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 30s 17ms/step - accuracy: 0.9327 - lo
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9739 - lo
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9806 - lo
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 17ms/step - accuracy: 0.9844 - lo
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 17ms/step - accuracy: 0.9866 - lo
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9886 - lo
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9901 - lo
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9914 - lo
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 30s 18ms/step - accuracy: 0.9930 - lo
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 18ms/step - accuracy: 0.9941 - lo
```

Accuracy vs Epochs (Batch=32)


Train Accuracy
Validation Accuracy
0.99
0.98
0.97
0.96
0.95
0.94
0.93
Accuracy
0
2
4
6
8
Epochs